



Software Testing and Quality Assurance

(05302438)

Yadagiri Rama Deepak,
Assistant Professor
Computer Science & Applications



Module-2

2.Levels of Testing-(W-15)

- Unit testing,
- Integration testing,
- System testing,
- Acceptance testing
- Alpha testing,
- beta testing,
- Static and dynamic testing,
- Manual and automation testing,
- Case study on testing.



Unit testing

Unit Testing is a type of software testing where individual components or functions of a program are tested in isolation from the rest of the application. Its primary goal is to validate that each unit of the software performs as expected.

Definition:

Unit testing involves testing smallest testable parts of an application — usually functions, methods, or classes — independently to ensure they work correctly.

Popular Unit Testing Frameworks:

Language	Framework
Java	JUnit
Python	Unit test / pytest
C++	Google Test
JavaScript	Jest / Mocha
C#	NUnit / MSTest

Benefits of Unit Testing:

Increases **code reliability**.

Enables **refactoring** without fear of breaking existing code.

Provides **documentation** of code behavior.

Supports **continuous integration** workflows.



Integration testing

Integration Testing is a level of software testing where individual units or modules are combined and tested as a group to evaluate how they interact with each other.

Definition:

Integration testing focuses on checking the **data flow and interactions between modules**, ensuring that the combined units work together correctly.

Key Objectives:

- Verify **interfaces** between components.
- Detect **interaction defects** between modules.
- Ensure **data exchange** between units is valid and consistent.

Integration Testing Approaches:

Approach	Description
Big Bang	All modules are combined and tested at once. Simple but hard to debug.
Top-Down	Testing starts from the top module and moves downward. Stubs are used for lower modules.
Bottom-Up	Starts from the lowest modules and moves upward. Drivers are used for higher modules.
Sandwich/Hybrid	Combination of Top-Down and Bottom-Up.



Benefits:

- Identifies **interface issues** early.
- Ensures that modules **work together properly**.
- Helps detect **data format mismatch**, API misuse, or logical errors across boundaries.



Challenges:

- Requires **test drivers/stubs**.
- Integration points can be **complex**.
- More **time-consuming** than unit testing.

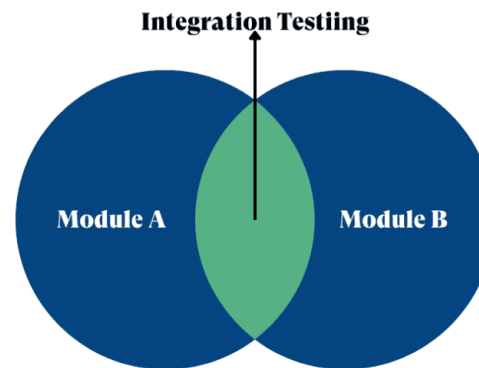
Example Scenario:

You have three modules in a banking app:

- **Login Module**
- **Transaction Module**
- **Balance Module**

After unit testing each, integration testing ensures:

- Login credentials pass correctly to the transaction module.
- The transaction module updates the balance module accurately.



System testing

System testing is Also known as end-to-end system testing ([E2E testing](#)) is a type of black-box testing that is done on a fully integrated application. Every input in the application is thoroughly verified for the desired output. System testing is the testing of a user's experiences with the application.

System Testing is the level of software testing where the **entire integrated system** is tested as a whole to ensure it meets the specified requirements.



Acceptance testing

Operational Acceptance Testing (OAT) is a type of **non-functional testing** done to check if a product is ready for real-world use.

It focuses on things like:

- **Recovery** (can the system recover from problems?),
- **Compatibility** (does it work well with other systems?),
- **Maintainability** (is it easy to update or fix?),
- **Reliability** (does it run without failure?).

.



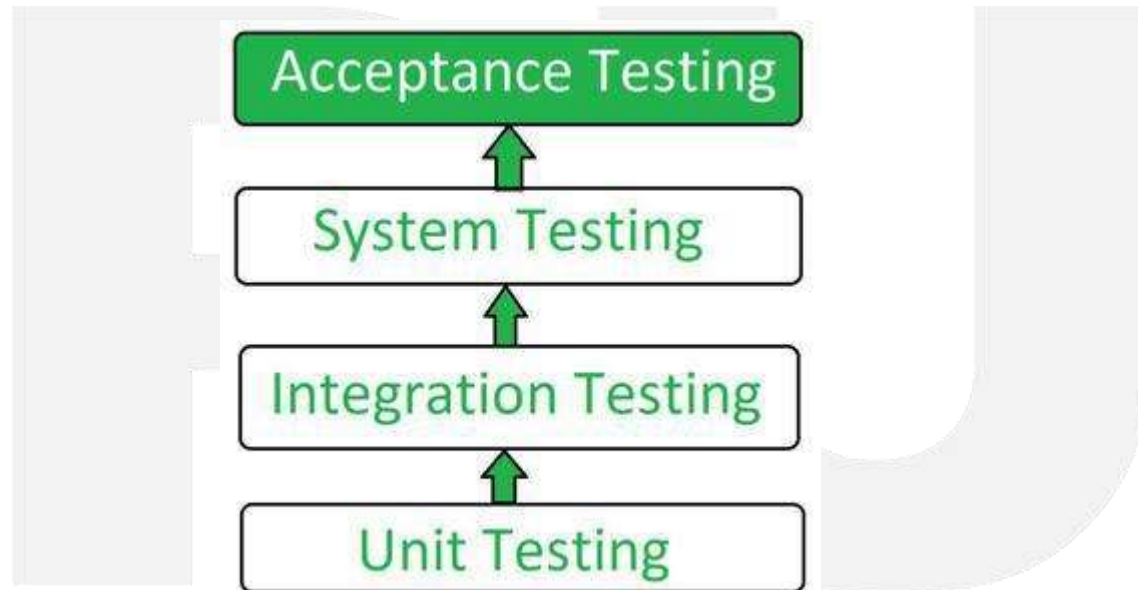
Acceptance testing

Acceptance Testing is the final stage of software testing. It checks if the system meets the **user's needs, requirements, and business processes**.

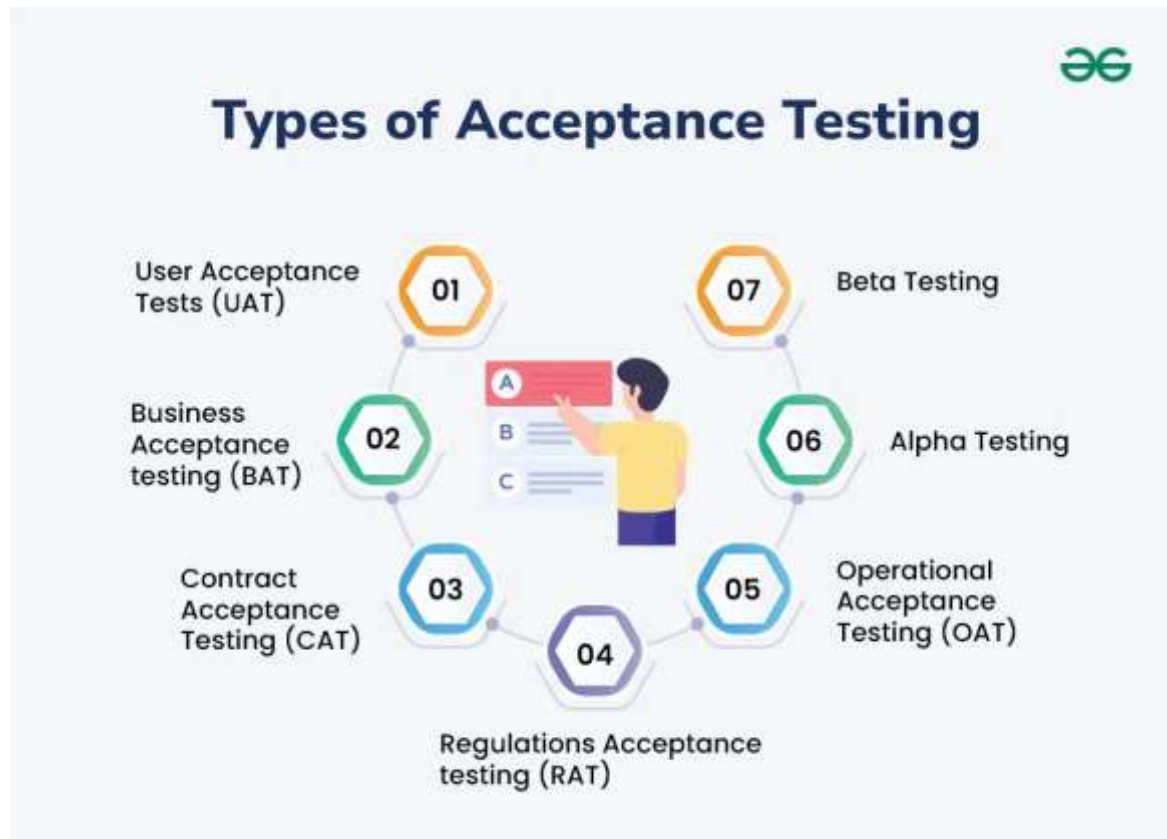
This testing helps users, customers, or other decision-makers decide **whether to accept the system or not**.

It is done **after system testing** and **before the software is used in the real world**.

Acceptance testing



Types of Acceptance Testing





Acceptance Testing

User Acceptance Testing (UAT)

- User acceptance testing is used to determine whether the product is working for the user correctly.
- Specific requirements which are quite often used by the customers are primarily picked for testing purposes. This is also termed as End-User Testing.



Acceptance Testing

Business Acceptance Testing (BAT)

- BAT is used to determine whether the product meets the business goals and purposes or not.
- BAT mainly focuses on business profits which are quite challenging due to the changing market conditions and new technologies, so the current implementation may have to be changed which results in extra budgets.



Acceptance Testing

Contract Acceptance Testing (CAT)

- CAT is a contract that specifies that once the product goes live, within a predetermined period, the acceptance test must be performed, and it should pass all the acceptance use cases.
- Here is a contract termed a Service Level Agreement (SLA), which includes the terms where the payment will be made only if the Product services are in-line with all the requirements, which means the contract is fulfilled.
- Sometimes, this contract happens before the product goes live.
- There should be a well-defined contract in terms of the period of testing, areas of testing, conditions on issues encountered at later stages, payments, etc.



Acceptance Testing

Regulations Acceptance Testing (RAT)

- RAT is used to determine whether the product violates the rules and regulations that are defined by the government of the country where it is being released.
- This may be unintentional but will impact negatively on the business. Generally, the product or application that is to be released in the market, has to go under RAT, as different countries or regions have different rules and regulations defined by its governing bodies.
- If any rules and regulations are violated for any country then that country or the specific region then the product will not be released in that country or region.
- If the product is released even though there is a violation then only the vendors of the product will be directly responsible.

Acceptance Testing

Operational Acceptance Testing (OAT)

- OAT is used to determine the operational readiness of the product and is non-functional testing.
- It mainly includes testing of recovery, compatibility, maintainability, reliability, etc. OAT assures the stability of the product before it is released to production.



Alpha testing

Alpha testing is an early phase of user acceptance testing where a software product is tested internally by a development or QA team before being released to external users. It focuses on identifying bugs, verifying core functionality, and ensuring the software meets its intended requirements in a controlled environment.

Alpha testing

Advantages of Alpha Testing

Early identification of bugs and issues: Alpha testing allows for the early identification of bugs and issues, providing an opportunity to fix them before they reach end-users.

Improved quality: By identifying and fixing bugs and issues early in the development process, alpha testing helps to improve the overall quality of the software.

Increased user satisfaction: Alpha testing helps to ensure that the software meets the needs of the target audience, leading to increased user satisfaction.

Faster resolution of problems: Alpha testing allows for the rapid resolution of problems, reducing the likelihood of further issues down the line.

Cost savings: By identifying and fixing issues early in the development process, alpha testing can help to save time and money by avoiding the need for more extensive testing and bug fixing later on.

Process of Alpha Testing

Process of Alpha Testing

Review the design specification and functional requirements.

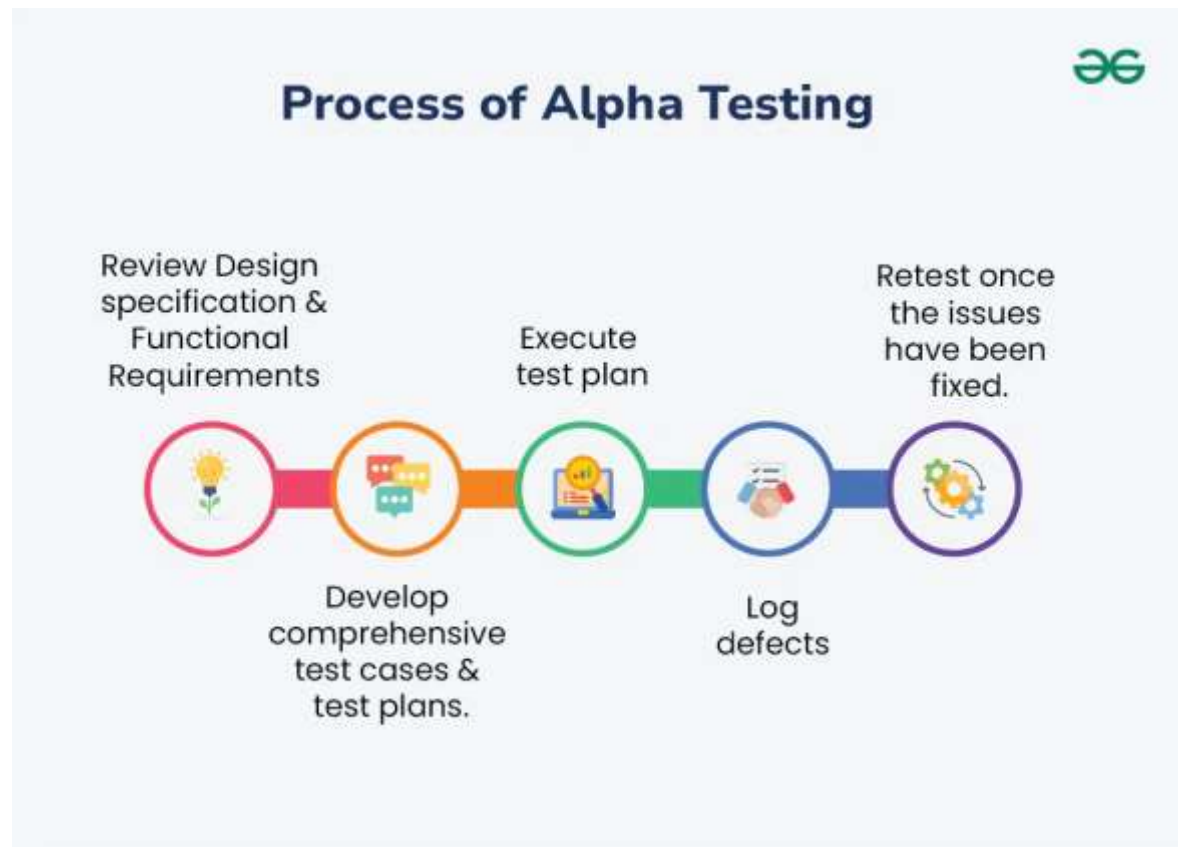
Develop comprehensive test cases and test plans.

Execute test plan

Log defects

Retest once the issues have been fixed.

Process of Alpha Testing





Phases of Alpha Testing

There are **two phases** in alpha testing:

1st Phase: The first phase of testing is done by in-house developers or software engineers. They either use hardware-aided debuggers or debugger software. The aim is to catch bugs quickly. Usually while alpha testing, a tester comes across to lots of bugs, crashes, missing features, and docs.

2nd Phase: The second phase of alpha testing is done by software quality assurance staff for additional testing in an environment. It includes a black box as well as white box testing.

Phases of Alpha Testing





Phases of Alpha Testing

1. **Planning:** This phase involves defining the scope, objectives, and schedule for the alpha testing process. It also includes identifying the target audience, the test environment, and the resources required for the testing.
2. **Preparation:** This phase involves setting up the test environment, configuring the test cases, and preparing the test data. It also includes creating the test scripts and building the test infrastructure.
3. **Execution:** This phase involves running the test cases and collecting the test results. Testers will report any bugs or issues they encounter, and the development team will work to fix them.
4. **Evaluation:** This phase involves analysing the test results and determining whether the software meets the requirements and performs as expected. It also includes identifying areas of improvement and making recommendations for further testing.
5. **Reporting:** This phase involves documenting the test results and providing a report to the development team and stakeholders. It also includes presenting the findings and recommendations for future testing and development.
6. **Closure:** This phase involves wrapping up the testing process and releasing the software for further testing or for release to the end-users.



Advantages of Alpha Testing

Alpha testing offers several advantages:

- **Early Bug Detection** : Identifies and addresses bugs early in the development process, reducing the risk of major issues later.
- **Improved Quality** : Enhances the overall quality and stability of the software before it reaches real users.
- **Cost-Effective** : Fixing issues during alpha testing is generally cheaper than addressing them after release.
- **Usability Insights** : Provides valuable feedback on the user experience, allowing for improvements in usability and interface design.
- **Requirement Validation** : Ensures the software meets business and user requirements, aligning it more closely with intended goals.
- **Controlled Environment** : Conducted in a controlled setting, allowing for thorough and systematic testing without external variables



Disadvantages of Alpha Testing

Alpha testing also has some disadvantages:

- **Limited Scope** : Conducted in a controlled environment, it may not cover all real-world scenarios and edge cases.
- **Internal Bias** : Performed by internal teams, who might have biases or overlook issues that external users would notice.
- **Time-Consuming** : Requires significant time and resources, potentially delaying the overall development timeline.
- **Incomplete Features** : At this stage, the software might still lack some features, making it harder to fully test the final product.
- **Resource Intensive** : Demands considerable effort from developers, QA teams, and sometimes other internal stakeholders.
- **False Sense of Security** : Successful alpha testing might create overconfidence, potentially leading to underestimation of issues during beta testing or after release.



Beta testing

Beta testing is a critical phase in product development, especially for software and applications, where real-world users test a product before its official release. This phase allows developers to gather feedback, identify bugs, and improve usability in a production environment. Beta testing is distinct from alpha testing, which is conducted internally by the development team.



Characteristics of Beta Testing

- Beta Testing is performed by clients or users who are not employees of the company.
- Reliability, security, and robustness are checked during beta testing.
- Beta Testing commonly uses black-box testing.
- Beta testing is carried out in the user's location.
- Beta testing doesn't require a lab or testing environment.



Types of Beta Testing

There are different types of beta testing:

1. **Traditional Beta testing:** Product is distributed to the target market and related data is gathered in all aspects. This data can be used for Product improvement.
2. **Public Beta Testing:** Product is released publicly to the world through online channels and data can be collected from anyone. Based on feedback, product improvements can be done. For example, Microsoft conducted the largest of all Beta Tests for its operating system Windows 8 before officially releasing it.
3. **Technical Beta Testing:** Product is released to a group of employees of an organization and collects feedback/data from the employees of the organization.
4. **Focused Beta Testing:** a Software product is released to the market for collecting feedback on specific features of the program. For example, important functionality of the software.
5. **Post-release Beta Testing:** a Software product is released to the market and data is collected to make improvements for the future release of the product.

Criteria for Beta Testing

1. Sign off a document on Alpha testing.
2. The Beta version of the software should be ready
3. Environment ready to release the software application to the public
4. Tool to capture real-time faults

Tools used for Beta Testing

1. Test Fairy
2. Center Code
3. TryMyUI
4. User Testing
5. TestRail
6. Usersnap
7. Zephyr
8. TestFlight



Advantages of Beta Testing

1. It reduces product failure risk via customer validation.
2. Beta Testing allows a company to test post-launch infrastructure.
3. It helps in improving product quality via customer feedback.
4. Cost-effective compared to similar data gathering methods.
5. It creates goodwill with customers and increases customer satisfaction.

Disadvantages of Beta Testing

1. Sometimes, it is complex to follow the errors or bugs because the testing environment varies from user to user.
2. There is a chance of having duplication of errors or bugs.
3. The development team and the testing team are not having control over this real-time test environment.
4. This testing is a time-consuming process since it involves real time users or clients and hence delay in the overall feedback about the entire product.
5. The users who are testing the product should have enough knowledge about the working of the entire application or product. Otherwise, the testing will not be implemented efficiently.

Static Testing

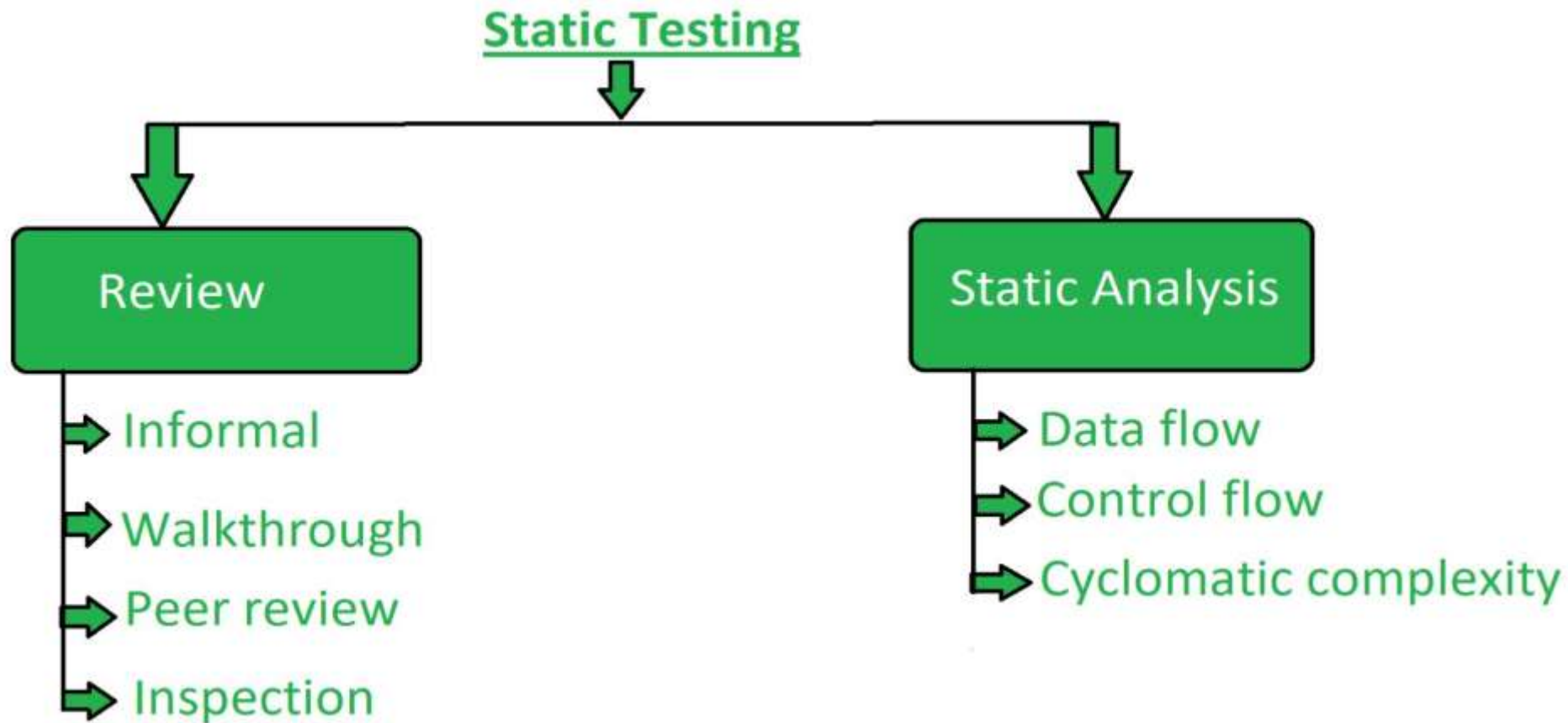
Static Testing is a type of Software Testing method that is performed to check the defects in software without actually executing the code of the software application. Whereas in Dynamic Testing checks, the code is executed to detect the defects. The article focuses on discussing Static Testing in detail.

What is Static Testing?

Static Testing also known as Verification testing or Non-execution testing is a type of Software Testing method that is performed in the early stage of development to avoid errors as it is easier to find sources of failures and it can be fixed easily. The errors that cannot be found using Dynamic Testing, can be easily found by Static Testing.

1. Static can be done manually or with the help of tools to find bugs and improve the quality of the software.
2. It helps to find errors in the early stage of development which is also called the verification process.
3. It enhances maintainability and ultimately saves time and money in the long run.

Static Testing Techniques





1.Review

In static testing, the review is a process or technique that is performed to find potential defects in the design of the software. It is a process to detect and remove errors and defects in the different supporting documents like software requirements specifications. People examine the documents and sorted out errors, redundancies, and ambiguities. Review is of four types:

1. **Informal:** In an informal review the creator of the documents put the contents in front of an audience and everyone gives their opinion and thus defects are identified in the early stage.
2. **Walkthrough:** It is basically performed by an experienced person or expert to check the defects so that there might not be problems further in the development or testing phase.
3. **Peer review:** Peer review means checking documents of one another to detect and fix defects. It is basically done in a team of colleagues.
4. **Inspection:** Inspection is basically the verification of documents by the higher authority like the verification of software requirement specifications (SRS).

2. Static Analysis

Static Analysis includes the evaluation of the code quality that is written by developers. Different tools are used to do the analysis of the code and comparison of the same with the standard. It also helps in following identification of the following defects:

1. Unused variables.
2. Dead code.
3. Infinite loops.
4. Variable with an undefined value.
5. Wrong syntax.

Static Analysis is of three types:

1. **Data Flow:** Data flow is related to the stream processing.
2. **Control Flow:** Control flow is basically how the statements or instructions are executed.
3. **Cyclomatic Complexity:** Cyclomatic complexity defines the number of independent paths in the control flow graph made from the code or flowchart so that a minimum number of test cases can be designed for each independent path.



Dynamic testing

Dynamic testing is a software testing method that involves executing the code and evaluating its behavior at runtime. It focuses on validating the functionality and performance of the software by providing input and verifying the output, unlike static testing which analyzes the code without execution. Dynamic testing is crucial for identifying runtime errors, performance bottlenecks, and security vulnerabilities that may not be apparent in static analysis.

Dynamic testing

Dynamic Testing is a software testing strategy used to test the dynamic conduct of software code. The fundamental motivation behind dynamic testing is to test software conduct with dynamic factors or factors which are not steady and discover feeble regions in software runtime climate.

Dynamic testing

DYNAMIC TESTING PROCESS



Dynamic testing

Benefits of Dynamic Software Testing

- Dynamic Testing can uncover the uncovered imperfections that are viewed as excessively troublesome or confounded and which can't be covered through static Analysis
- In Dynamic Testing, we execute the software, start to finish, guaranteeing blunder free software which thus builds the nature of an item and venture.
- Dynamic Testing turns into a fundamental Tool for recognizing any security Threats.

Manual and Automation testing

What is Manual Testing?

Manual testing is a type of testing in which we do not use any tools (automation) to perform the testing. In this testing, testers make test cases for the codes test the software, and give the final report about that software. Manual testing is time-consuming testing because humans do it and there is a chance of human errors.

Manual testing is conducted to discover bugs in the developed software application.

Manual and Automation testing

- The tester checks all the essential features of the application.
- The tester executes test cases and generates test reports without any help from the automation tools.
- It is conducted by the experienced tester to accomplish the testing process.

Manual and Automation testing

When to Perform Manual Testing?

Manual testing is done when automation can't be used or isn't enough. Here are key reasons and scenarios:

- **Exploratory Testing:** Discovering issues in new or unclear features by exploring them.
- **Usability Testing:** Checking if the UI is easy to use and looks good.
- **Ad-Hoc Testing:** Doing quick, informal tests after updates or bug fixes.
- **Visual/GUI Testing:** Checking the layout, colors, or how the site looks across different browsers.

Manual and Automation testing

Benefits of Manual Testing

Here are the Benefits of Manual Testing:

- **Easy hiring:** In manual testing, anyone can test so it helps in easy hiring.
- **Fast feedback:** Manual testing helps to provide fast and accurate feedback.
- **Versatile:** Manual test cases can be applied to many test cases.
- **Flexible:** Manual testing is flexible as it can adapt easily to changes in the user interface.
- **Less expensive:** Manual testing is less expensive as one does not need to spend a budget on automation tools and processes.



Manual and Automation testing

What is Automation Testing?

Automation testing is a type of testing in which we take the help of tools (automation) to perform the testing. It is faster than manual testing because it is done with some automation tools. There is no chance of any human errors.

- It relies entirely on pre-scripted test which runs automatically to compare actual results with expected results.
- Automation testing helps the tester determine whether the application performs as expected or not.
- It allows the execution of repetitive tasks and regression tests.
- Automation requires manual effort to create initial testing scripts.

Manual and Automation testing

When to Perform Automation Testing?

When need to run repetitive tasks: Automated tests are the best option in scenarios where there is a requirement to run repetitive tests. For example, in the case of regression tests must be executed periodically to make sure that the newly added code does not disrupt the existing functionality of the software.

When human resources are scarce: Automated tests are viable and the best option to get tests executed within deadlines when there are only a limited number of dedicated testers.



Manual and Automation testing

Benefits of Automation Testing

Here are Benefits of Automation Testing:

- **Finds more bugs:** Automation testing helps to find more bugs and defects in the software.
- **Reduce time for regression tests:** Automated tests are suitable for regression tests as the tests can be executed in a repetitive manner periodically.
- **The process can be recorded:** This is one of the benefits of using automation tests as these tests can be recorded and thus allows to reuse of the tests.
- **No fatigue:** As automation, tests are executed using software tools so there is no fatigue or tiring factor as in manual testing.
- **Increased test coverage:** Automation tests help to increase the test coverage as using the tool for testing helps to make sure that not even the smallest unit is left for testing.



Difference Between Manual Testing and Automated Testing

Parameters	Manual Testing	Automation Testing
Definition	In manual testing, the test cases are executed by the human tester.	In automated testing, the test cases are executed by the software tools.
Processing Time	Manual testing is time-consuming.	Automation testing is faster than manual testing.
Resources requirement	Manual testing takes up human resources.	Automation testing takes up automation tools and trained employees.
Exploratory testing	Exploratory testing is possible in manual testing.	Exploratory testing is not possible in automation testing.
Framework requirement	Manual testing doesn't use frameworks.	Automation testing uses frameworks like Data Driven, Keyword, etc.
Reliability	Manual testing is not reliable due to the possibility of manual errors.	Automated testing is more reliable due to the use of automated tools and scripts.
Investment	In manual testing, investment is required for human resources.	In automated testing, investment is required for tools and automated

Case Study

Company Overview:

Name: ShopEase Ltd. (*fictional*)

Industry: E-Commerce

Platform: Web and mobile application for online shopping

Users: ~500,000 active users/month

Tech Stack: React (Frontend), Node.js (Backend), MongoDB (Database)

Project Goal:

Launch a new feature – "**Smart Cart**", which applies personalized discounts, real-time stock validation, and predictive suggestions.



Challenges:

- Multiple integrations (Payment gateways, Inventory system, User profiles)
- Tight release deadline (4 weeks)
- Frequent UI changes
- Security concerns due to payment handling
- Need for cross-browser and mobile responsiveness



Testing Objectives:

Testing Objectives:

- Ensure the new feature works across all platforms
- Validate payment and discount calculations
- Detect any regressions in the core cart functionality
- Ensure the user interface is responsive and bug-free
- Ensure data integrity and security for transactions

Testing Process:

◆ 1. Test Planning

Prepared a **Test Strategy Document**

Identified **test cases**, tools, team, and timelines

Chose **Agile methodology** with **2-week sprints**



Testing Process:

Type	Description	Tools Used
Unit Testing	Tested discount logic, stock validator module	Jest, Mocha
Integration Testing	Tested cart and payment gateway interactions	Postman, REST Assured
System Testing	Full user journey (product → cart → payment)	Selenium, TestNG
UI/UX Testing	Cross-browser and device testing	BrowserStack
Security Testing	Ensured no SQL injection, secure payment flow	OWASP ZAP
Performance Testing	Checked response time under load	JMeter

3. Test Case Design:

- Used **Equivalence Partitioning** and **Boundary Value Analysis**
- Created **450+ test cases**
 - 60 Unit
 - 150 Integration
 - 140 Functional
 - 40 Security
 - 60 Performance

Results:

Metric	Value
Total Test Cases Executed	450
Bugs Detected	89
Critical Bugs Fixed	12
Regression Bugs	14
Time Taken	3.5 weeks
Automation Coverage Achieved	75%
Post-release Defects	2 minor UI bugs

The testing team at ShopEase successfully validated and delivered the **Smart Cart feature** within the deadline. With effective **manual + automated testing, cross-functional coordination**, and **early defect detection**, they minimized post-release defects and ensured a smooth user experience.

Attachments (Available on Request):

Test Plan Sample

Bug Report Snapshot

Automation Test Script (Selenium)

JIRA Dashboard Screenshot

× ○ DIGITAL LEARNING CONTENT



Parul[®] University



www.paruluniversity.ac.in

